

---

# **Implementation of Fully Connected Feedforward Network in C++ to Run Pretrained Models**

---

**SHAHRIAR RAHMAN DIPON**

Submitted to: Dr. Xin Chen

2024-11-18

# Contents

|          |                                                                                |            |
|----------|--------------------------------------------------------------------------------|------------|
| <b>1</b> | <b>Abstract</b>                                                                | <b>1</b>   |
| <b>2</b> | <b>Introduction</b>                                                            | <b>1</b>   |
| <b>3</b> | <b>Background</b>                                                              | <b>2</b>   |
| 3.1      | Flow* . . . . .                                                                | 2          |
| 3.2      | ONNX File Format . . . . .                                                     | 2          |
| <b>4</b> | <b>Class Design</b>                                                            | <b>2</b>   |
| 4.1      | Layer . . . . .                                                                | 3          |
| 4.1.1    | Function Name: Layer . . . . .                                                 | 3          |
| 4.1.2    | Function Name: Relu . . . . .                                                  | 3          |
| 4.1.3    | Function Name: forward . . . . .                                               | 4          |
| 4.1.4    | Function Name: Softmax . . . . .                                               | 4          |
| 4.1.5    | Function Name: Tanh . . . . .                                                  | 5          |
| 4.2      | FCNetwork . . . . .                                                            | 6          |
| 4.2.1    | Function Name: FCNetwork . . . . .                                             | 6          |
| <b>5</b> | <b>Class Interaction</b>                                                       | <b>6</b>   |
| <b>6</b> | <b>Implementation</b>                                                          | <b>7</b>   |
| <b>7</b> | <b>Results</b>                                                                 | <b>10</b>  |
| 7.1      | Small Sized Networks . . . . .                                                 | 10         |
| 7.2      | Test on Large Sized Network . . . . .                                          | 14         |
| <b>8</b> | <b>Limitations</b>                                                             | <b>15</b>  |
| 8.1      | Compilation Issues with ONNX Runtime in C++ . . . . .                          | 15         |
| 8.2      | Parsing ONNX Graph Nodes and Operations . . . . .                              | 15         |
| 8.3      | Challenges and Adjustments in Model Testing with ONNX and PyTorch . . . . .    | 16         |
| 8.4      | Handling Activation Functions in ONNX Parsing and C++ Implementation . . . . . | 17         |
| 8.5      | Parsing ONNX Model with GEMM Nodes and Identity Mapping . . . . .              | 17         |
| <b>9</b> | <b>Conclusion</b>                                                              | <b>17</b>  |
|          | <b>List of Figures</b>                                                         | <b>I</b>   |
|          | <b>List of Tables</b>                                                          | <b>II</b>  |
|          | <b>Nomenclature</b>                                                            | <b>III</b> |

**10 Appendix A**

**III**

---

# 1 Abstract

## Abstract

In this work, we present a C++ implementation of a fully connected neural network designed for efficient forward propagation, aimed at use cases where rapid execution is crucial, such as in control systems. While deep learning models have proven highly effective in classification and regression tasks, their heavy reliance on memory and processing power has traditionally limited their use in real-time applications. With advances in processing capabilities and memory, implementing and testing these models has become feasible even in performance-critical systems.

Our implementation enables models trained in popular deep learning frameworks, such as PyTorch, to be saved in the ONNX format and loaded into a C++ environment. A custom Python script parses the ONNX file to extract model architecture and parameters, which are then used to create an equivalent neural network structure in C++. We validated the correctness of our implementation with test cases comparing model performance in both Python and C++ environments. This approach extends the usability of neural network models beyond Python's interpreted runtime, supporting their integration into systems requiring machine-level execution. More details on the implementation and testing can be found on our GitHub repository.

## 2 Introduction

Neural networks architectures have outperformed machine learning algorithms in many classification and regression tasks. Most deep learning models need an exponential amount of memory with respect to the input data. In the past, testing these models were not feasible. Currently, with the availability of powerful processing units and large memory space, implementing and testing deep learning models have become feasible.

Deep learning models have outperformed general machine learning algorithms like SVM, and K-Nearest Neighbors. Despite being memory and processor hungry, deep learning models are able to solve problems across different domains. There is ongoing research on how to improve their performance across different aspects.

However, many systems require fast processing during runtime. A lot of the implementations are made in Python programming language. Python is an interpreted language which requires it to convert the code into machine code line-by-line at runtime by the Python interpreter. Since the code is translated during execution, its performance is slow. For this reason many applications of Python implementations is limited. Trained neural network models can be used as part of control systems that require fast execution.

Control systems will require models to execute faster at machine level in the system. To extend the usability of trained neural network models in controlled systems, we have implemented fully connected networks in C++ for forward propagation.

---

Models trained in deep learning libraries including PyTorch and Tensorflow can be saved in ONNX format. Our implementation takes in the ONNX file as input, derive the neural network architecture along with the parameters, and then create an instance of the same neural network model in C++.

## 3 Background

### 3.1 Flow\*

Flow\* is a toolbox designed for modeling and analysing cyber-physical systems (CPS). It primarily focuses on computing reachable set overapproximations that guarantee the inclusion of all approximation and numerical errors. The toolbox can handle discrete, continuous, and hybrid dynamics over bounded time intervals. It is implemented in C++.

In our implementation, we have used the Matrix library from Flow\*. The `Matrix` template class type in the Flow\* library was used extensively in our implementation for binary and unary operations involving matrices.

### 3.2 ONNX File Format

ONNX (Open Neural Network Exchange) is an open-source format designed to represent machine learning models. It allows models to be shared across different frameworks and platforms, making it easier to develop and deploy machine learning applications.

## 4 Class Design

Our neural network implementation primarily consisted of two components- the **layer** and the **net**.

The layer is a unit of the hidden class in neural network and it is implemented in the **Layer** class.

The **net** consists of all the hidden layers and the output layer of the neural network. It should be noticed that the net consists of multiple components of layer. The **net** component is implemented in the **FCNetwork** class.

The **net** consists of all the hidden layers and the output layer of the neural network. It should be noticed that the net consists of multiple components of layer. The **net** component is implemented in the **Net** class.

## 4.1 Layer

The Layer class was implemented to denote a single layer in fully connected neural network. After initialization of the weight, bias, and activation function, an object of the layer is able to perform the forward propagation operation for a given input.

Inside the layer class, multiple functions are implemented to carry out the task of forward propagation. Table lists all the implemented functions along with their descriptions

### 4.1.1 Function Name: Layer

**Description:** Constructs a Layer object by initializing weights, bias, and the activation function type.

**Inputs:**

- `Matrix<double> weights_`: The matrix containing the weights for the layer.
- `Matrix<double> bias_`: The matrix containing the bias for the layer.
- `int sigma_`: The activation function identifier (0 for Identity, 1 for ReLU, 14 for Softmax).

**Outputs:**

- N/A (Constructor does not return a value).

**Function Signature:**

```
Layer(Matrix<double> weights_, Matrix<double> bias_, int sigma_);
```

**Example Usage:**

```
// Initialize weight and bias matrices
Matrix<double> weights(3, 3);
Matrix<double> bias(3, 1);

// Create a Layer object with the weights, bias, and activation function type
Layer layer(weights, bias, 1); // 1 indicates ReLU activation function
```

### 4.1.2 Function Name: Relu

**Description Inputs:**

- `Matrix<double> a`: The input matrix to apply the Rectified Linear Activation Unite (ReLU) function.

**Output:**

- `Matrix<double>`: The output matrix with ReLU applied element-wise.

### 4.1.3 Function Name: forward

**Description:** Performs the forward pass through the layer, applying the linear transformation followed by the activation function.

**Inputs:**

- `Matrix<double> X_`: The input matrix for the forward pass.
- `bool transpose`: If true, transposes the input matrix before processing.

**Outputs:**

- `Matrix<double>`: The output matrix after applying the linear transformation and activation function.

**Function Signature:**

```
Matrix<double> forward(Matrix<double> X_, bool transpose=false);
```

**Example Usage:**

```
Matrix<double> input(3, 1);
input[0][0] = 1.0;
input[1][0] = 2.0;
input[2][0] = 3.0;

Layer layer(weights, bias, sigma);
Matrix<double> output = layer.forward(input);
```

### 4.1.4 Function Name: Softmax

**Description** Applies the Softmax function to the input matrix to normalize the output to a probability distribution.

**Inputs:**

- `Matrix<double> a`: The input matrix to apply the Softmax function.
- `bool return_proba`: If true, returns the probabilities; otherwise, returns raw values.

**Outputs:**

- `Matrix<double>`: The output matrix after applying the Softmax function.

**Function Signature:**

```
Matrix<double> Softmax(Matrix<double> a, bool return_proba=true);
```

**Example Usage:**

```
// Initialize input matrix
Matrix<double> input(3, 1);
input[0][0] = 2.0;
input[1][0] = 1.0;
input[2][0] = 0.1;

// Apply Softmax
Matrix<double> softmax_output = layer.Softmax(input);
```

Function Name: Sigmoid **Description** Applies the Sigmoid activation function to the input matrix element-wise.

**Inputs:**

- Matrix<double> a: The input matrix to apply the Sigmoid function.

**Outputs:**

- Matrix<double>: The output matrix with Sigmoid applied element-wise.

**Function Signature:**

```
Matrix<double> Sigmoid(Matrix<double> a);
```

**Example Usage:**

```
// Initialize input matrix
Matrix<double> input(3, 1);
input[0][0] = -1.0;
input[1][0] = 0.0;
input[2][0] = 1.0;

// Apply Sigmoid
Matrix<double> sigmoid_output = layer.Sigmoid(input);
```

**4.1.5 Function Name: Tanh**

**Description** Applies the hyperbolic tangent activation function to the input matrix element-wise.

**Inputs:**

- Matrix<double> a: The input matrix to apply the Tanh function.

**Outputs:**



- `Matrix<double>`: The output matrix with Tanh applied element-wise.

**Function Signature:**

```
Matrix<double> Tanh(Matrix<double> a);
```

**Example Usage:**

```
// Initialize input matrix
Matrix<double> input(3, 1);
input[0][0] = -1.0;
input[1][0] = 0.0;
input[2][0] = 1.0;

// Apply Tanh
Matrix<double> tanh_output = layer.Tanh(input);
```

## 4.2 FCNetwork

The FCNetwork class takes an ONNX file name as the parameters. In the project directory, the ONNX file must be stored in the model folder. An instance of the network object will take the filename- not the complete directory of the file as parameter. It will read the data structure of the file, store the weights, biases, and activations in a text file in the `weights_and_biases` folder in the project directory. Weights and biases are read from the text file and instance of Layer objects are created which are pushed to the end of Layer type vector. This concatenation of Layer objects duplication the fully connected neural network passed in the ONNX file.

### 4.2.1 Function Name: FCNetwork

**Description Inputs:**

- `Matrix<double> a`: The input matrix to apply the Rectified Linear Activation Unit (ReLU) function.

**Output:**

- `Matrix<double>`: The output matrix with ReLU applied element-wise.

## 5 Class Interaction

In the network class, we pass the name ONNX file that we want to parse. We assume that the ONNX file passed as parameter is a fully connected neural network model that is saved in ONNX

---

format after training. The ONNX file format arranges neural network architecture in a graph-like structure. Each node of the graph is derived from a layer unit in the neural network. The edges denote the connectivity between the adjacent layers.

We have a Python script that parses the ONNX file to decipher the data structure of the neural network and retrieve the weights and bias values in each of the layers of the graph. This Python script is called inside the Network constructor function. This script is named `gemm_parser_v1.py` in our project folder.

After successful execution of the script, we store the weights, bias, activation function values from the ONNX file and stored it in a text file in the `weights_and_biases` folder in our project directory. The naming convention adopted was the name of the ONNX file (before the file extension) concatenated with `weights_and_biases.txt`.

After storing the information of the neural network in a text file, the network constructor function opens and reads through the text file and extract information for the layers of the neural network. Information on the weights, bias, and activation code information are stored in the text file in sequence according to the architecture the neural network. Figure shows how one layer is stored in the text file.

This pattern of storing information is repeated in the file for consecutive layers. The activation code maps an activation function to an integer.

After information on the weights, biases, and activation function is retrieved from a layer, an instance of Layer object is created which is then push into vector of Layer. Consecutive reading of the layers from the text file, creating a Layer object, and then pushing it onto a vector of Layer object confirms we have the neural network from the ONNX format stored in an instance of Network object type.

## 6 Implementation

To incorporate the Matrix class and its operators from the Flow\* library, we first retrieved the Flow\* library from the official page. Within our local setup, the Flow\* library is organized in a directory named `flowstar`. Inside this, we navigated to the `/benchmarks/continuous` folder, where we created a dedicated folder for our new project.

We retrieved the Flow\* library from the official github repository. In our local machine, the Flow\* library is in a folder named `flowstar`. We navigate to the `/benchmarks/continuous` folder where we created a dedicated folder for our project.

To use the Matrix class and its operators in `flow*` for our implementation, we have retrieved `flow*` library from the official page. Inside the project `flow*` project directory `/benchmarks/continuous`, we have created our project folder.

- **5\_models\_and\_data\_samples.ipynb** - A Jupyter Notebook that contains five models and associated data samples used for training and evaluation.
- **compare\_results\_of\_models.ipynb** - A Jupyter Notebook for comparing the results of different models based on performance metrics.

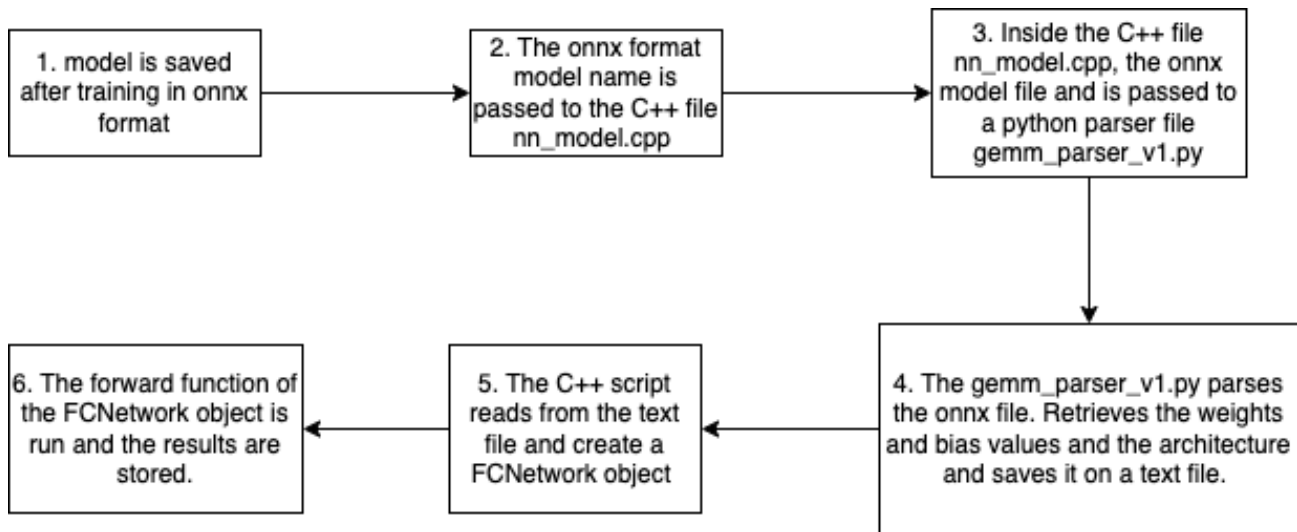


Figure 1: Interaction of Different Components

- **gemm\_parser\_v1.py** - Parses an ONNX file with gemm operator and retrieves and stores the information in a text file. The ONNX file must be in `model\ONNX` directory and only the name of the ONNX file should be passed rather than the complete directory.
- **data** - Stores the randomly generated samples for test
- **model** - Includes the models in two subdirectories **ONNX** and **Python** ; The **ONNX** folder stores the ONNX format models and **torch** stores pth format models.
- **results** - A directory or file that stores the results of model evaluations or training sessions.
- **weights\_and\_biases** - The model architecture is stored in this directory after parsing.
- **Makefile** - A Makefile that automates the build process for the project.
- **layer.h** - Contains the declared member variables and methods for a single layer the neural network.
- **layer.cpp** - Contains the implementations of the functions declared in `layer.h`.
- **fc\_network.h** - Contains the declared memeber variables and methods for implementing a neural network comprising of multiple layers.
- **fc\_network.cpp** - Contains the imlementations of the functions declared in `fc_network.h`.
- **nn\_model.cpp** - Script to test the 5 networks for sine function with our implementation.
- **regression.cpp** - Script to test the 5 networks for large regression models with our implementation.

- 
- **regression\_model.ipynb** - Python notebook with script for instantiating X layer neural network followed by training and saving the model, and then storing the result of one forward pass of trained model on test data.
  - **run\_ONNX\_in\_Python.sh** - Shell script to execute `gemm_parser_v1.py`

The file structure in the project directory is given below for clarification.

---

```
flowstar
  benchmarks
    continuous
      fc_network
        model
          onnx
          pth
        data
        results
          onnx
          pth
          error_rate.txt
        weights_and_biases
        regression_model_3.ipynb
        regression_model_4.ipynb
        regression_model_5.ipynb
        regression_model_6.ipynb
        regression_model_7.ipynb
        5_models_and_data_samples.ipynb
        compare_results_of_models.ipynb
        compare_results_of_regression_models.ipynb/
        gemm_parser_v1.py
        Makefile
        layer.cpp
        layer.h
        fc_network.cpp
        fc_network.h
        nn_model.cpp
        regression.cpp
        run_ONNX_in_Python.sh
```

## 7 Results

### 7.1 Small Sized Networks

We tested our implementation of fully connected neural network by running 5 pretrained We have checked the correctness of our implementation as follows: i) We create 5 models in Pytorch to approximate the sine function. ii) We train each the of the models on randomly 1000 randomly generated function and their sine functions. iii) We save the model in .pth and .ONNX format after

training from Python notebook file in model.pth and model/ONNX iv) We do test of our model by reading on model instances for each model in PyTorch by reading the weights and biases from .pth file saved in model.pth v) We test our C++ implementation of fully connected forward network. We read the file from model/ONNX subdirectory and save the results in result/ONNX.

1. We create 5 models in PyTorch to approximate the sine function. Each model is of different depth and different number of layers per unit.
2. We train each of the models on 1000 randomly generated function inputs and their corresponding sine function values.
3. After training, we save the models in both .pth and .ONNX formats from Python notebook file in the model.pth and model/ONNX directories.
4. We test our models by loading model instances for each model in PyTorch, reading the weights and biases from the .pth files saved in model.pth.
5. We test our C++ implementation of a fully connected forward network. We read the files from the model/ONNX subdirectory and save the results in results/ONNX.
6. We compare the saved models in pth and ONNX Step 4 and Step 5 format to check the correctness of our implementation in C++.

Step 1, 2, 3, and 4 are obtained by running `5_models_and_data_samples.ipynb` notebook.

Step 5 is obtained by running `nn_model.cpp`.

Step 6 is obtained by running `compare_results_of_models.ipynb`.

We save the results from Step 4 and Step 5 in `results/pth` and `results/ONNX` respectively.

The results from step 6 in results in `error_rate.txt` file.

After training, the predictions made by each of the trained model on the training set is as follows:

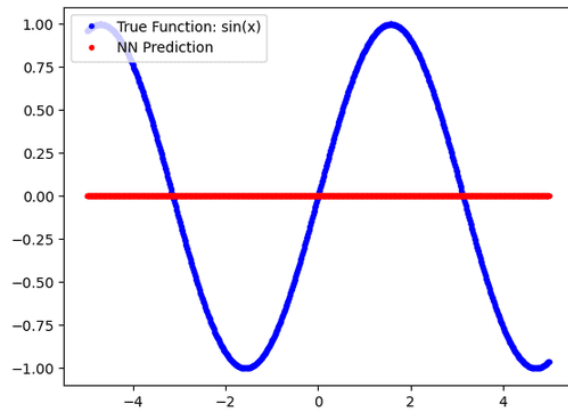


Figure 2: Network 1

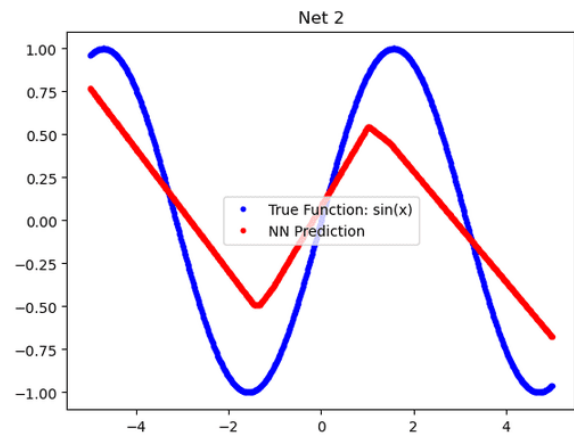


Figure 3: Network 2

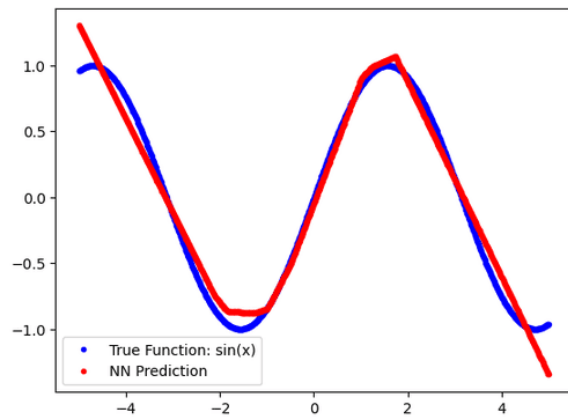


Figure 4: Network 3

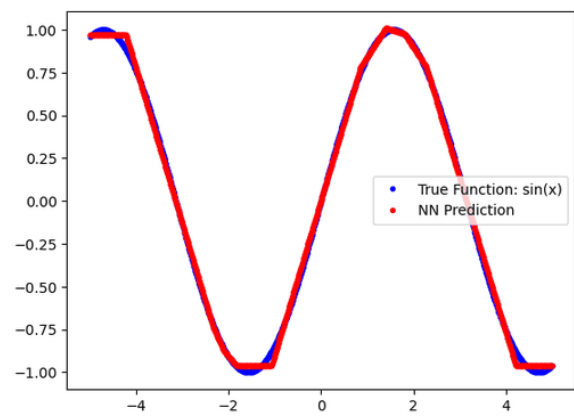


Figure 5: Network 4

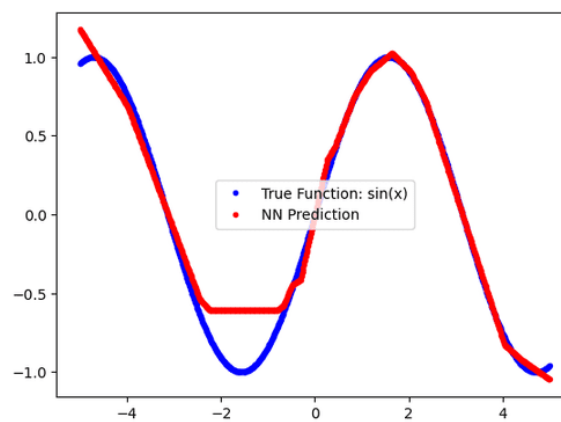


Figure 6: Network 5

We tested each of the models 5 sets of 20 datasets. We load the saved pretrained model from Step 1-5 in the model directory in our project folder. We load our pth file extension in Python environment, test the model on 100 test cases, and save the results in results/pth. Likewise, we run our nn\_model.cpp which reads each of the 5 models in ONNX format, saves their weights, biases and activations in the weights\_and\_biases folder, creates fully connected neural network instances from the stored information, runs each models on the 100 randomly generated test cases across 5 files, and stores the results in results/ONNX subdirectory.

At step 6, we run compare\_results\_and\_models.ipynb which calculates and the difference between the two sets of results for 100 test cases and saves them result in error\_rate.txt file results.

Our aim was to keep the error threshold at most to the order of  $10^{-7}$ . Below is the result for model 4 across 5 datasets. pth and ONNX format. On the table below, we show the absolute difference between for a pretrained model 4 on a set of 20 test cases.

The table below shows the absolute difference for model 4 save in ONNX and pth format for a set of 20 test cases.

| Pth (in PyTorch)     | Onnx Format (our C++ imlemenation) | Absolute Difference    |
|----------------------|------------------------------------|------------------------|
| -0.3863258361816406  | -0.386325830328017                 | 5.853623641360883e-09  |
| -0.9188283085823059  | -0.9188282286890437                | 7.989326222812565e-08  |
| -0.4701603055000305  | -0.4701604038158445                | 9.8315813990002e-08    |
| -1.074212670326233   | -1.074212862053513                 | 1.9172728005223405e-07 |
| -0.09589849412441254 | -0.09589848871502374               | 5.4093887980055655e-09 |
| 0.9063632488250732   | 0.9063633700429957                 | 1.2121792247743457e-07 |
| 0.93610680103302     | 0.9361069858545201                 | 1.848215001043485e-07  |
| 0.990511953830719    | 0.9905117732795499                 | 1.8055116912663038e-07 |
| 1.0042014122009277   | 1.004201423348529                  | 1.1147601286509712e-08 |
| -0.6658962368965149  | -0.6658961971811193                | 3.971539563885074e-08  |
| 0.9170011281967163   | 0.9170012308635938                 | 1.0266687744397984e-07 |
| 0.5637996196746826   | 0.5637993048485684                 | 3.1482611417832373e-07 |
| -1.189522624015808   | -1.189522537889962                 | 8.612584601053186e-08  |
| -0.9252412915229797  | -0.9252412055332497                | 8.598973000317756e-08  |
| 0.5079097151756287   | 0.5079096512275942                 | 6.394803442333341e-08  |
| 1.0067682266235352   | 1.006768266926398                  | 4.0302862824859176e-08 |
| 0.07647143304347992  | 0.07647145738165043                | 2.433817050673248e-08  |
| -0.708229124546051   | -0.7082291591026166                | 3.455656560902298e-08  |
| 0.09188735485076904  | 0.091887296769634                  | 5.808113504035539e-08  |
| -0.4523026943206787  | -0.4523027279891533                | 3.3668474563874895e-08 |

Table 1: Absolute difference between for a pretrained model 4 on a set of 20 test cases.



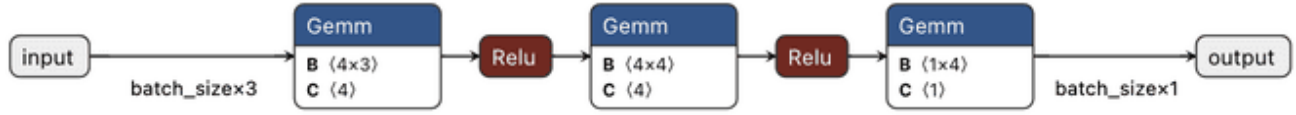


Figure 7: Neural Network with 3 layers



Figure 8: Neural Network with 4 layers

Results for more test cases can be found in the `results/error_rate.txt` in the github repository of the project.

## 7.2 Test on Large Sized Network

To further test the correctness of our implementation, we tested our implementation on neural network models of different size from 3 layer to 7 layer in increasing order. With the sci-kit learn library we generated 2000 instances of data that included 1500 instances used for training and 500 instances used for testing. Each tuple had three features the target label is a real number. Each of our five neural network model was trained on 1500 instances of training data and 500 instances of test data. We follow the same procedures as in the previous section, and compare the results.

Table 2 shows the first 20 predictions made by our neural network model for regression that has a 7 layers. We see that for neural network on increasing size and complexity, the absolute difference in the outcomes of the two implementations has increased. Previously, for simpler models and small number of data, the absolute error was at most in the order of  $10^7$ . With increasing model complexity, we see that the absolute error is now in the order of  $10^6$ .

For other models, we have noticed a similar pattern. We have encountered many cases where the absolute difference in error was in the order of  $10^5$ . Multiple attempts were made to address this, but the issue persisted.

We deduced that this was due to the increased complexity in our test cases that induced rounding error. When a value was read to initialize parameters, steps were taken to ensure that rounding-off error could be avoided as much as possible. For example, every value was read precisely from 17-20 decimal places. While these measurements ensured our outcome remained within the desired error threshold for simpler designs, the same measure improved the results for larger and more complex

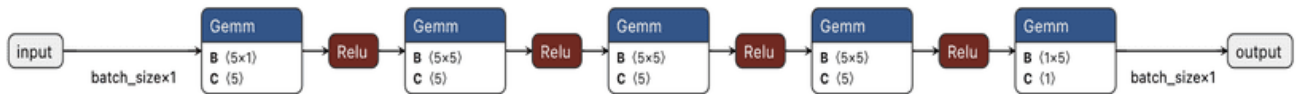


Figure 9: Neural Network with 5 layers



Figure 10: Neural Network with 6 layers

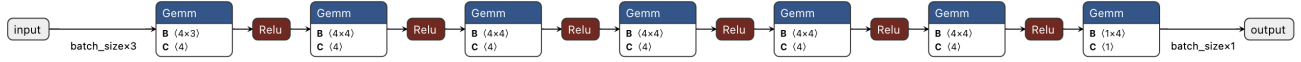


Figure 11: Neural Network with 7 layers

network but not to the extent we aimed for.

We see that the model trains well. We further compare our implementation in C++ and python script that takes in an onnx against the pytorch model that takes in a pth file. Table 2 shows our the difference in absolute error for the first 20 instances of test set.

We see that the absolute difference in the predicted value is still marginal although the at the bare minimum the absolute difference starts off from the order of  $10^{-4}$ . This is a significant increase in error compared to small networks.

The results for other models are similar and can be found on the GitHub repository.

## 8 Limitations

### 8.1 Compilation Issues with ONNX Runtime in C++

We attempted to use the ONNXruntime library for C++ to parse the models stored in ONNX file. Library for ONNXruntime in C++ is available which was downloaded and tried on local machine. We could not run the library on the local machine and faced compilation error. After many attempts to address this, we decided to write down the parser in Python with available libraries for ONNX file parsing.

### 8.2 Parsing ONNX Graph Nodes and Operations

In an ONNX file, weights and biases are stored as nodes along with their values. Mathematical operations are performed between two nodes and the resultant tensor is stored in a third node. An input propagates along the graph until the result is returned.

From the documentations, we have found that two operations are more frequently used than others- the MatMul (Matrix Multiplication) and Gemm (General Matrix Multiplication). An ONNX graph uses either one of the two operators.

The MatMul performs general binary operation between two nodes. Our initial parser was implemented with the to parse ONNX graphs with MatMul operator. We tested our operator across multiple yet simple test cases and observed that our parser could successfully parse graphs with MatMul operator.

| Pth (in PyTorch)    | Onnx Format (our C++ implementation) | Absolute Difference    |
|---------------------|--------------------------------------|------------------------|
| 46.86872482299805   | 46.868725643140372                   | 6.302807605607086e-06  |
| 62.40884780883789   | 62.408845339387341                   | 1.0237822891667747e-06 |
| 81.068603515625     | 81.068600532505499                   | 6.725316978872797e-07  |
| -39.68339538574219  | -39.683380672539101                  | 6.329031840834887e-06  |
| -42.208160400390625 | -42.208162672006068                  | 1.6589416702572635e-06 |
| 67.93246459960938   | 67.932476162448566                   | 7.728579078047915e-06  |
| 15.070630073547363  | 15.070628147067964                   | 2.5481142316152727e-06 |
| 30.93402671813965   | 30.93403463365285                    | 2.1792475308757275e-06 |
| -17.966350555419922 | -17.966324902237105                  | 1.0822161531365282e-05 |
| 120.94721221923828  | 120.94721011122834                   | 3.958416968430356e-06  |
| 75.50567626953125   | 75.505691670410357                   | 7.754398993142786e-06  |
| 3.2081422805786133  | 3.2081360726543799                   | 4.701073599733263e-06  |
| 2.139852285385132   | 2.1398492688935455                   | 4.533389740934979e-06  |
| 55.08309555053711   | 55.083088371331833                   | 3.7297203618891217e-06 |
| 16.775606155395508  | 16.775604822075767                   | 1.4905644789564576e-06 |
| -66.14510345458984  | -66.145057793735276                  | 3.1242047100477066e-06 |
| 19.072275161743164  | 19.072271930973333                   | 6.248850681345175e-08  |
| -86.47954559326172  | -86.479541568913675                  | 1.574854908881207e-05  |
| 74.92317962646484   | 74.923192372458729                   | 3.207318314935037e-06  |
| -9.870519638061523  | -9.8705055864545415                  | 4.844482628385549e-07  |

Table 2: Absolute difference between Pth (in PyTorch) and Onnx format (our C++ implementation) for a pretrained model on a set of 20 test cases for 7-layer neural network.

However, during further testing it was found out that PyTorch always use Gemm operator when they save a model in ONNX format. Which is why we had to implement a separate parser specifically for the Gemm operator.

### 8.3 Challenges and Adjustments in Model Testing with ONNX and PyTorch

After training, we saved our trained model in two separate files- a .pth file and a .ONNX file. For testing purpose, initially we wanted to use the .ONNX file to test in our environment for Python and for C++. However, We faced challenges loading the ONNX file into a model instance of Torch. Which is why during testing we chose saved and instantiated the same model in two format for two different environment. We later tried to load the ONNX model with the ONNXruntime library. After facing consistent error with the size of the input- which was bizarre given that the input dimension was the same of either test and train cases, we decided to save models after training in .pth format and then load it onto an instance of the model created with PyTorch.

## 8.4 Handling Activation Functions in ONNX Parsing and C++ Implementation

Our neural network accounts for a handful of activation functions in use and these are ReLU, tanh, sigmoid, softmax. While the parser script in Python would be able to parse different activation function of a layer in ONNX file, we may not be able to create model instances for all of them in our C++ implementation of neural network.

## 8.5 Parsing ONNX Model with GEMM Nodes and Identity Mapping

The Python script `gemm_parser_v1.py` parses an ONNX file and stores the information of the neural network in a text file. The parser takes into account of the the presence of the weight, bias, and activation node in each layer of the model in ONNX graph. For some models, we have seen that there are no activation layers after the linear operation. In such cases, the parser would assume that it is a case of identity mapping and document the presence of an identity mapping.

## 9 Conclusion

We have implemented a C++ implementation of fully connected neural network. Our results show that our implementation correctly parses the neural network model save in ONNX format with a Python script, and that our C++ implementation can create a data structure from the saved information from parsing. The correctness of our implementation is validated by the test cases we have for our models. We further test our implementation for large neural networks which still returns formidable results. More information can be found on the GitHub repository.

## List of Figures

|    |                                               |    |
|----|-----------------------------------------------|----|
| 1  | Interaction of Different Components . . . . . | 8  |
| 2  | Network 1 . . . . .                           | 12 |
| 3  | Network 2 . . . . .                           | 12 |
| 4  | Network 3 . . . . .                           | 12 |
| 5  | Network 4 . . . . .                           | 12 |
| 6  | Network 5 . . . . .                           | 12 |
| 7  | Neural Network with 3 layers . . . . .        | 14 |
| 8  | Neural Network with 4 layers . . . . .        | 14 |
| 9  | Neural Network with 5 layers . . . . .        | 14 |
| 10 | Neural Network with 6 layers . . . . .        | 15 |
| 11 | Neural Network with 7 layers . . . . .        | 15 |

## List of Tables

|   |                                                                                                                                                                        |    |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Absolute difference between for a pretrained model 4 on a set of 20 test cases. . . .                                                                                  | 13 |
| 2 | Absolute difference between Pth (in PyTorch) and Onnx format (our C++ implementation) for a pretrained model on a set of 20 test cases for 7-layer neural network. . . | 16 |

---

## 10 Appendix A